Call by value vs Call by Reference

07 March 2026 20:14

In C programming, the primary difference between call by value and call by reference lies in how arguments are passed to a function and whether the original variable can be modified.

Call by Value

Pin call by value method. A copy of the actual variables value is passed to the function's formal parameter.

- **Memory:** The original variable and functions parameters are stored in two separate memory locations.
- **Modification:** Changes made to the parameter inside the function do not affect the original variable in the calling function.
- **Use Case:** This method is used when you want to protect the original data from accidental modification and is the default method for function calls in C language.

```c
#include<stdio.h>
void func(int num){
    num = num + 10;
    printf("\n%d", num);
}
int main(int argc, char const *argv[])
{
    int actual_num = 5;
    func(actual_num);
    printf("\nactual_num = %d", actual_num);
    return 0;
}
```

local variable of func()

when the code of func() function is over then 'num' is deleted from the memory.

copied to this num

Output:

```
15
actual_num = 5
```

Call by reference

Instead of value, the memory address of the variable is passed to the function.

- **Memory:** Both the actual and formal argument and the formal parameter. (which is a pointer) refer to the same memory location.
- **Modification:** Changes made to the value at the pointer's address inside the function directly affect the original variable in the calling function.
- **Use case:** This method is used when you need a function to modify the original variable or passing large data structure (like arrays or structs) to avoid the performance penalty of copying.

```c
#include<stdio.h>
void func(int *num){
    *num = *num + 10;
```

```c
#include<stdio.h>
void func(int *num){
    *num = *num + 10;
    printf("\n%d", *num);
}
int main(int argc, char const *argv[])
{
    int actual_num = 5;
    func(&actual_num);
    printf("\nactual_num = %d", actual_num);
    return 0;
}
```

int actual_num 15
1000

int num 1000

Output:

```
15
actual_num = 15
```

Difference

Feature	Call by value(pass by value)	Call by Reference(pass by reference)
Data passed	A copy of the value.	The memory address of the variable.
Original data	Not affected by function changes.	Directly affected by function changes.
Memory	Uses separate memory for the copy.	Uses the same memory location.
Syntax	Normal variable parameters.	Pointer parameters(*) and address of operator(&).
Efficiency	Slower for large data structures due to copying.	More efficient for large data structures.
Safety	Safer, prevents unintended modification.	Less safe, requires careful handling to avoid bugs.